

Yes. What you're describing is actually an important gap in system-design learning: **building something is not enough; you need a feedback loop that challenges your design.**

For your 10 side quests, I would use **four levels of validation**:

1. Self-review against a design rubric

Before coding, write:

- Functional requirements
- NFRs
- Expected scale
- Architecture
- Data model
- API contracts
- Consistency guarantees
- Failure scenarios
- Scaling strategy
- Trade-offs
- What you intentionally did **not** build

Then attack your own design:

“What happens if this component dies here?”

“What if two requests arrive simultaneously?”

“What if the network times out but the operation actually succeeded?”

“What happens when this becomes 10× larger?”

“Can this produce duplicates?”

“Where is the source of truth?”

This is essentially how you should train for interviews.

2. Get an external expert review

This is the part you're looking for.

There are communities where experienced engineers can critique architecture/design, but the quality varies. More importantly, you can make the review **very structured**.

Instead of asking:

“Is my architecture good?”

ask:

“Here is my requirement, architecture, assumptions and failure model. Please identify correctness issues, hidden race conditions, scaling bottlenecks and questionable trade-offs.”

That gives reviewers something concrete to attack.

Good places include:

- **r/ExperiencedDevs** — useful for experienced-engineer perspectives
- **r/softwarearchitecture** — architecture-focused discussion
- **r/systemdesign** — system-design-oriented discussions
- **GitHub** — put the architecture document and implementation publicly and ask for targeted review
- Your professional network / senior engineers — probably the highest-value feedback if you can get someone experienced in distributed systems to spend 20–30 minutes reviewing one design.

For your particular goal, I'd value **one experienced distributed-systems/backend engineer reviewing a design** more than collecting dozens of casual comments.

3. Use an adversarial design review

This is something I can help you do particularly well.

For example, suppose you build your **distributed rate limiter**.

Don't just ask me:

“Review my architecture.”

Give me your design document and say:

Act as a senior distributed-systems engineer conducting a hostile architecture review. Do not redesign it for me. Find correctness bugs, race conditions, failure modes, scalability problems and unjustified assumptions. Ask me questions one at a time and wait for my answer.

Then I can challenge you like an interviewer/reviewer.

For example:

You:

"I use Redis with a token bucket."

Reviewer:

"Two application instances can receive requests for the same user simultaneously. Show me how your implementation guarantees that both don't consume the same token."

You answer.

Then:

"What happens if Redis succeeds in consuming the token but the response to your application times out?"

You answer.

Then:

"Now Redis is unavailable. Do you reject traffic or allow it?"

You explain your decision.

Then:

"Why?"

That's much more valuable than me simply saying *"your architecture looks good."*

4. Validate the implementation against the design

This is the part many system-design learners skip.

Your project should have **failure tests**, not just unit tests.

For example, your webhook project should deliberately test:

```
Worker receives webhook
    ↓
sends HTTP request
    ↓
destination processes request
    ↓
network timeout
    ↓
worker doesn't know whether request succeeded
```

Then ask:

Does my system create a duplicate?

Likewise:

Project	Important failure test
Rate limiter	Concurrent requests hitting same key
Idempotency	Same request arriving 100 times concurrently
Webhook	Destination succeeds but response times out
Job queue	Worker crashes after claiming job
Cache	10,000 requests hit an expired hot key
Distributed lock	Lock holder dies before releasing
Workflow	Worker dies halfway through a DAG
CDC	Consumer crashes after processing but before checkpoint
Event broker	Consumer crashes after processing message
KV store	Replica becomes unavailable during write

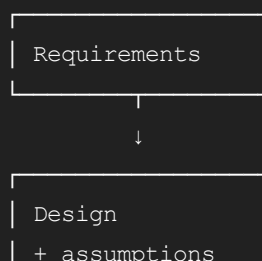
This turns your project from “**I implemented Redis/Kafka**” into:

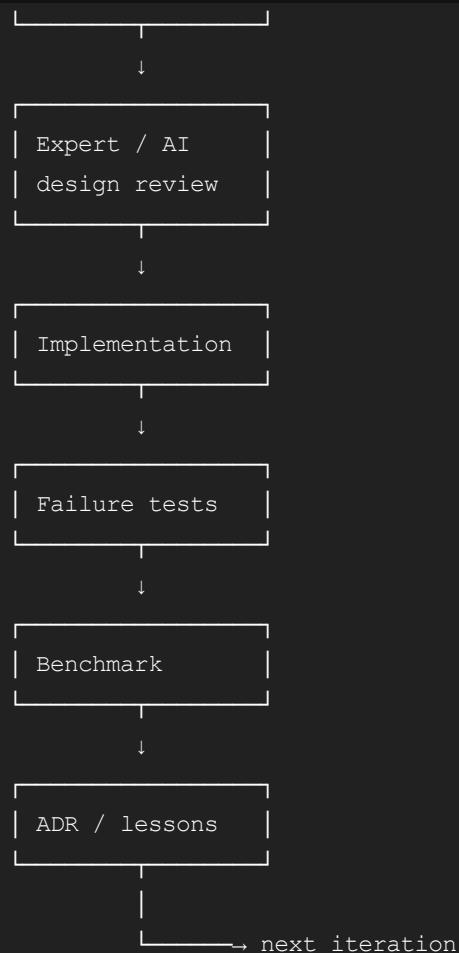
“I investigated what happens when distributed components fail.”

That's much stronger learning.

I would actually create a validation loop for every one of your 10 projects

Something like:





And there's an important distinction

Don't ask an expert to tell you **the correct architecture**.

For distributed systems, there often isn't one.

You want them to tell you:

- **What assumption is wrong?**
- **What failure case did I miss?**
- **Where is my design inconsistent?**
- **What happens under concurrency?**
- **What happens under partial failure?**
- **What doesn't scale?**
- **Which trade-off did I make without realizing it?**
- **What would I be asked in a senior-level interview?**

That's the feedback that will actually improve your system-design ability.

And we can make this a formal part of your 10-project plan: **each project gets a Requirements Review → Architecture Review → Adversarial Review → Failure Testing → Postmortem.**

That would make the projects substantially more valuable than simply putting ten GitHub repositories on your résumé.